Stellar Cyber 6.6.0s is now live! Discover all the updates in the release notes!

# Configuring API Authentication 6.2.0

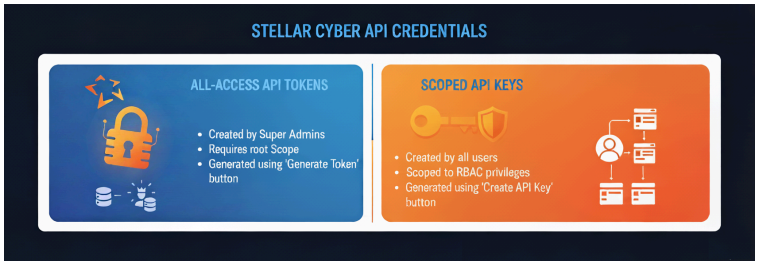
This topic describes the authentication required to make calls to the Stellar Cyber API. In general, you will need a user account with sufficient privileges, an API token or key, and a JSON Web Token created from the API token or key. See the following sections for details:

- **Understanding the Two Types of API Credentials in Stellar Cyber**
- **About Scoped API Keys**
- **Restricted Public API Endpoints**
- **Required Privileges to Make API Calls**
- **Necessary Information to Make API Calls**

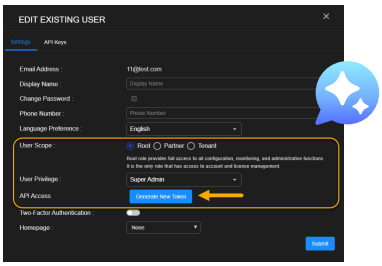
If you are trying to authenticate with the API test page found under **? | API Docs**, refer to **Authenticating with the API Test Page** for instructions.

## Understanding the Two Types of API Credentials in Stellar Cyber 6.2.0

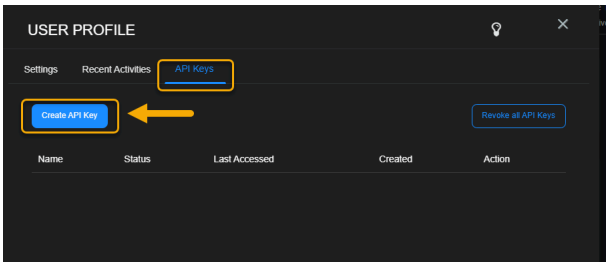
As illustrated below, Stellar Cyber uses two types of API credentials, *tokens* and *keys*.




- **All-Access API tokens** – **Super Admins** with **root** Scope can create an API **token** using the **Generate Token** button in the **API Access** section of the Edit User dialog box. API tokens were available before the 5.5.0 release and are still available in the current release. They provide access to all public API endpoints, including some restricted endpoints that are not available using scoped API keys at all.

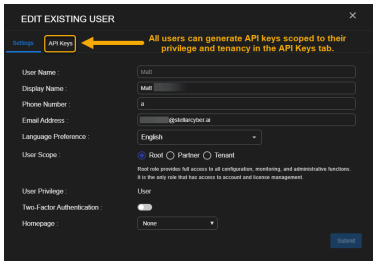



- **Scoped API keys** – **All users** can generate an API key scoped to their RBAC privileges and tenancy by clicking the **Create API Key** button in the **API Keys** tab of the Edit User and User Profile dialog boxes. Scoped API keys were introduced in the 5.5.0 release.




# About Scoped API Keys 5.5.0

Prior to the 5.5.0 release, public API access was only available to users with **Root** scope and **Super Admin** privileges. Releases from 5.5.0 onwards complement that support with per-user API keys, scoped to a user's RBAC privileges and tenancy. You can now configure API keys for your own account, regardless of your scope and privileges in the **API Keys** tab in the Edit User and User Profile dialog boxes, as illustrated below:




## Benefits of Scoped API Keys

This implementation of API keys has the following benefits:

- Respects the industry-standard principle of least privilege, only granting an API key the tools that it needs to perform the job you want it to do.
- Allows easy breach isolation, with API keys tied to specific users.
- Enables immediate key revocation to contain exposures quickly and limit the dwell time of any unauthorized access.

Stellar Cyber recommends that you consider creating service accounts scoped with only the RBAC privileges necessary to perform specific tasks. This way you can grant personnel just the access they need to perform specific API tasks.

## Managing Scoped API Keys

Both the Edit User and User Profile dialog boxes include an **API Keys** tab that lets you create and revoke an account's API keys. The keys listed in the **API Keys** tab are specific to the selected account and are limited by its user scope (tenancy) and RBAC privileges:

- An account can only make calls to the public API endpoints available to its assigned RBAC privileges.

  Certain public API endpoints are restricted to **Super Admin** users with **root** scope. Refer to **Restricted Public API Endpoints** for a list.

- An account can only access API data available to its user scope and tenancy. For example:
  - A partner user can access data for any of its associated tenants.
  - A tenant user can only access data for its own tenancy.
  - A root user can access data from the entire platform.

You use the API keys in the **API Keys** tab to generate a JSON Web Token for access to the API.

Administrative users can revoke keys for users with lower privileges than their own, subject to the normal limits on tenancy and scope. They cannot, however, create a key for another user.

| Version Number | Account Requirements | Authentication Available |
|---|---|---|
| 5.4.0 | Super Admin users with Root scope | |
| 5.4.0s | Super Admin users with Root scope | |

## Restricted Public API Endpoints 6.1.0

The table below lists the public API endpoints that remain accessible only to **Super Admin** users with **root** scope using all-access API tokens. These endpoints do not support user-scoped access:

| Version 5.5.0+ | Version 5.5.0s+ |
|---|---|
| /storage-usages | /storage-usages |
| /storage-usages/daily_and_monthly | /storage-usages/daily_and_monthly |
| /incidents | /incidents |
| /entity_usages/entity_list/{scope} | /entity_usages/entity_list/{scope} |
| /reset_user | /reset_user |
| /ingestion-stats | /data |
| /data | Detections \| Coverage Analyzer (https://coverage-analyzer.stellarcyber.cloud/) |

| Version 5.5.0+ | Version 5.5.0s+ |
| --- | --- |
| /update_ser | |
| /security_events/{index}/[id\|tags\|status\|comment] | |
| /insert_ser | |
| /insert_ser_bulk | |
| Detections \| Coverage Analyzer (https://coverage-analyzer.stellarcyber.cloud/) | |

# Required Privileges to Make API Calls

There are no user account requirements to make API calls. The available public API endpoints are limited by account's RBAC privileges and tenancy.

We recommend creating a Stellar Cyber user dedicated to API calls. That way you can easily track changes made through API calls under **System | ORGANIZATION MANAGEMENT | Users | Activity Log**. You can create multiple service accounts, each dedicated to a particular level of API access.

Access to the API is only available using local user accounts. Single-sign on (SSO) users cannot access the API with their SSO credentials.

# Necessary Information to Make API Calls

Calls to the Stellar Cyber API typically require a subset of the following information:

| Data | Description |
| --- | --- |
| **Email Address** | Email address of the admin account making the call. |
| **API Key or Token** | Generate an API key or token as follows:<br>1. Navigate to **System \| ORGANIZATION MANAGEMENT \| Users**.<br>2. Locate the user account to perform the API call and select **Edit** (✎) in its row.<br>3. Use one of the following techniques, depending on your account privileges and scope:<br>• **API Keys (All Users)** – Click on the **API Keys** tab and use the options there to generate a new scoped key.<br>• **All-Access API Tokens (Super Admin users with root scope)** – Locate the **API Access** item in the dialog box that appears and select **Generate New Token**. You must be a Super Admin with root scope to use this technique.<br>4. Copy and paste the key or token into a text file to store it temporarily. |
| **YourStellarCyberServer** | The URL or IP address of your Stellar Cyber server. |

## Generating a JSON Web Token (JWT) from the API Key

Requests to the Stellar Cyber public APIs are secured using JSON Web Token (JWT) authentication.

**Click here to read more about JWT authentication, including how to use your API key to generate one on-demand.**

JWT authentication offers rapid, scalable, and secure authentication for API calls. They are also time-sensitive, expiring t minutes after they are generated. Because of this, scripts you write to pull data from the Stellar Cyber API should typically include a section used to refresh the requesting user's JWT.

The sample scripts in this topic include a section that refreshes the JWT using the supplied API Key.

You generate a JWT by sending a request to the public **https://<YourStellarCyberServer>/connect/api/v1/access_token** API on the DP. The request must include the API Key you created in the **API Keys** tab in either the Edit User or User Profile dialog box . The API Key acts as a refresh token that you use to generate a new JWT.

For example, the sample Python script below retrieves a JWT and prints it to the screen using the following details:

- **DP** – myserver.stellarcyber.cloud

  This is specified in the **HOST** variable at the top of the sample script.

- **API Key from GUI** – eyJhbGciOiJNisI1NiIsInR5cCI6IkpXVCJ9.eyJhcGlfa2V5Ijp7ImtleV9pZCI6ImU5MTRlZCIsInVzZXJfaWQiOiJjN2JkMDI3ZGEzMjc0MDU1YTZkODQ5YTk5N2QxYmUwOCIsIm9yZ19pZCI6IjQ5MGI2MDk2Yjk5ZTQ1ZDY4NTdjOTdjNDA4YjQzMTE4IiwiZXhwIjo0ODk3NzI0MzY3fSwiaWF0IjoxNzQ0MTI0MzY3fQ.ReZlP7DQB4w1UyVw3X1hwppFM4T3lZBxZviHfnVj2Sk

  This is specified in the **refresh_token** variable in the sample script.

```
#!/usr/bin/python3

import requests
import base64
import json
from urllib.parse import urlunparse
requests.packages.urllib3.disable_warnings()

# Step 1
# Add DP IP/hostname and refresh token from GUI here
HOST = "myserver.stellarcyber.cloud"
refresh_token =
"eyJhbGciOiJNisI1NiIsInR5cCI6IkpXVCJ9.eyJhcGlfa2V5Ijp7ImtleV9pZCI6ImU5MTRlZCIsInVz
ZXJfaWQiOiJjN2JkMDI3ZGEzMjc0MDU1YTZkODQ5YTk5N2QxYmUwOCIsIm9yZ19pZCI6IjQ5MGI2MDk2Yj
k5ZTQ1ZDY4NTdjOTdjNDA4YjQzMTE4IiwiZXhwIjo0ODk3NzI0MzY3fSwiaWF0IjoxNzQ0MTI0MzY3fQ.R
eZlP7DQB4w1UyVw3X1hwppFM4T3lZBxZviHfnVj2Sk"

def getAccessToken(refresh_token):
    headers = {
        "Authorization": "Bearer " + refresh_token
    }
    url = urlunparse(("https", HOST, "/connect/api/v1/access_token", "", "", ""))
    res = requests.post(url, headers=headers, verify=False)
    print(res.status_code)
    return res.json()["access_token"]

if __name__ == "__main__":

    # Step 2: Use getAccessToken with supplied credentials to generate JWT
    jwt = getAccessToken(refresh_token)
    print("------------ jwt -------------")
    print(jwt)
    print("------------ jwt  end -------------")
```

## Generating a New JWT as Part of a Script

Because JWTs expire ten minutes after they are generated, you may want to include logic in your scripts similar to the previous code sample to generate and use a fresh JWT every time the script is run.

For example, the code sample below uses the same **host** and **refresh_token** as above but includes it as part of a script that pulls the ten most severe cases for a specific tenant. Note the following:

- The script sets the **host** and **refresh_token** in the same places as the previous script (**Step 1** in the sample).
- The script runs the **getAccessToken** procedure to generate the new JWT (**Step 2** in the sample).
- The script uses the generated JWT to make a call to the cases API in the **getCases** procedure (**Step 3** in the sample). This procedure also includes specific arguments that specify the **tenant_id** and how many cases to retrieve (**10**), as well as the order in which to return them (**desc**).
- The script also prints the generated JWT to the screen. This, however, is not strictly necessary since the **getAccessToken** procedure already prints the status code for the call to the **access_token** API (**200** for success; **401** for failure).

```
#!/usr/bin/python3

import requests
import base64
import json
from urllib.parse import urlunparse
requests.packages.urllib3.disable_warnings()

# Step 1
# Add DP IP/hostname and refresh token from GUI here
HOST = "myserver.stellarcyber.cloud"
refresh_token =
"eyJhbGciOiJNisI1NiIsInR5cCI6IkpXVCJ9.eyJhcGlfa2V5Ijp7ImtleV9pZCI6ImU5MTRlZCIsInVzZX
JfaWQiOiJjN2JkMDI3ZGEzMjc0MDU1YTZkODQ5YTk5N2QxYmUwOCIsIm9yZ19pZCI6IjQ5MGI2MDk2Yjk5ZT
Q1ZDY4NTdjOTdjNDA4YjQzMTE4IiwiZXhwIjo0ODk3NzI0MzY3fSwiaWF0IjoxNzQ0MTI0MzY3fQ.ReZlP7D
QB4w1UyVw3X1hwppFM4T3lZBxZviHfnVj2Sk"

def getAccessToken(refresh_token):
    headers = {
        "Authorization": "Bearer " + refresh_token
    }
    url = urlunparse(("https", HOST, "/connect/api/v1/access_token", "", "", ""))
    res = requests.post(url, headers=headers, verify=False)
    print(res.status_code)
    return res.json()["access_token"]


def getCases(token):
    headers = {"Authorization": "Bearer " + token}
    url = urlunparse(("https", HOST, "/connect/api/v1/cases?
tenantid=817418af76d44358922636e34be9627c&limit=10&sort=score&order=desc", "", "",
""))
    res = requests.get(url, headers=headers, verify=False)
    print(res.status_code)
    return res.json()

if __name__ == "__main__":

    # Step 2: Use getAccessToken with supplied credentials to generate JWT
    jwt = getAccessToken(refresh_token)
    print("------------ jwt -------------")
    print(jwt)
    print("------------ jwt  end -------------")

    # Step 3: use JWT token to call public API
    cases = getCases(jwt)
    print("------------ call result of /connect/api/v1/cases -------------")
```

```
    print(cases)
    print("------------ end api results -------------")
```

# All API Calls Stored in User Activity Log

Any API request to the public APIs listed here is logged to the User Activity Log with the corresponding user. The request body is also visible by clicking the JSON Data button for the call in the User Activity Log page.



© 2026 Stellar Cyber . All rights reserved.